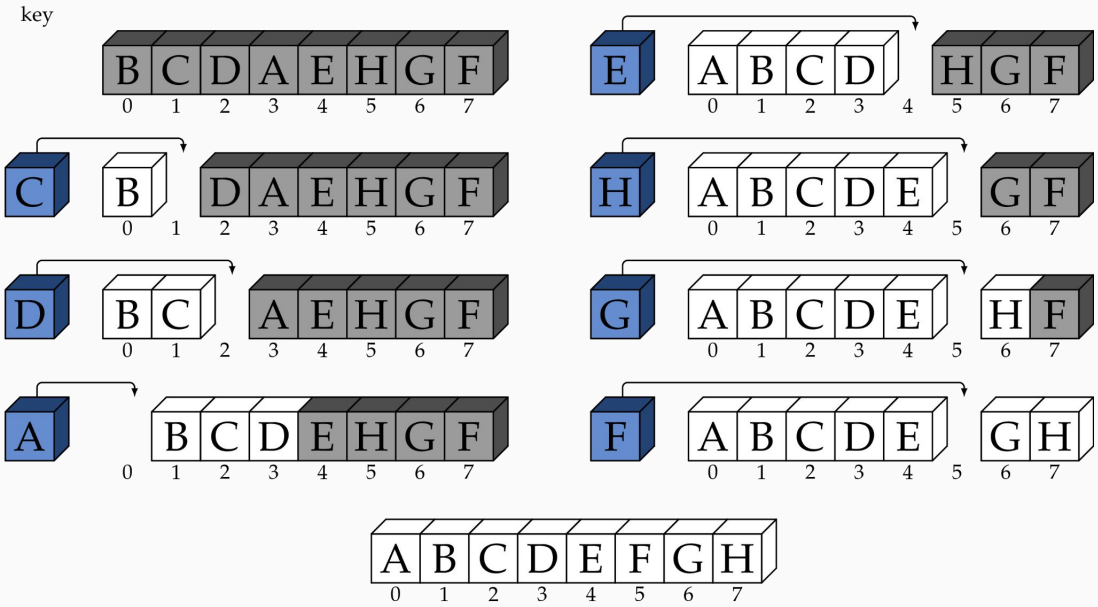
CSC 510 - Analysis of Algorithms

Prof. Timothy Sun

Insertion Sort

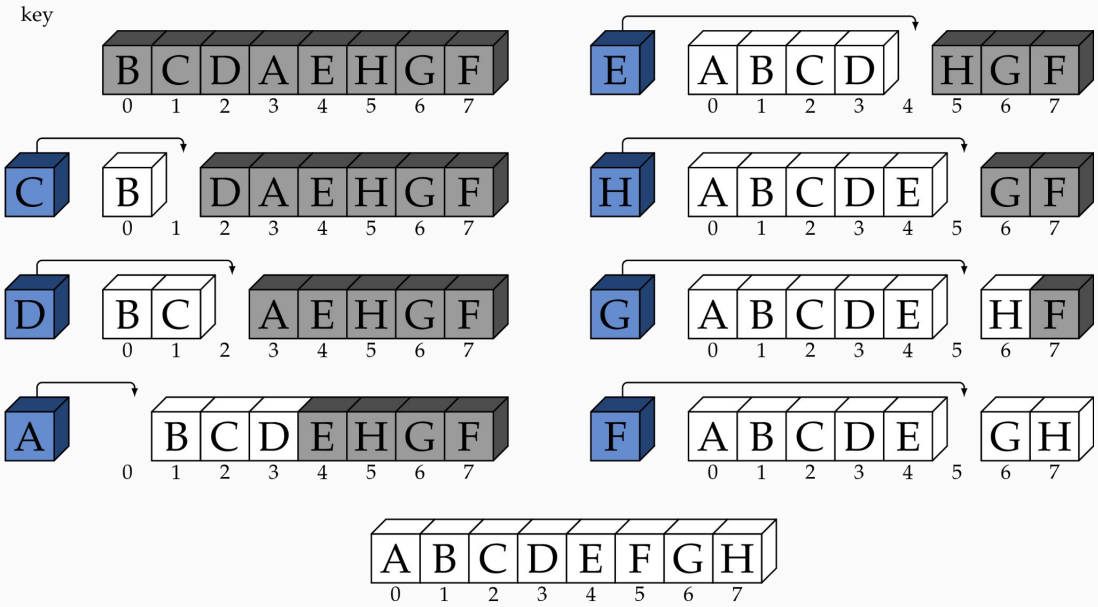
"At every step, look at the next object and **insert** it into its correct spot in the sorted sequence you've got so far."



Insertion Sort

"At every step, look at the next object and **insert** it into its correct spot in the sorted sequence you've got so far."

Is this an algorithm?

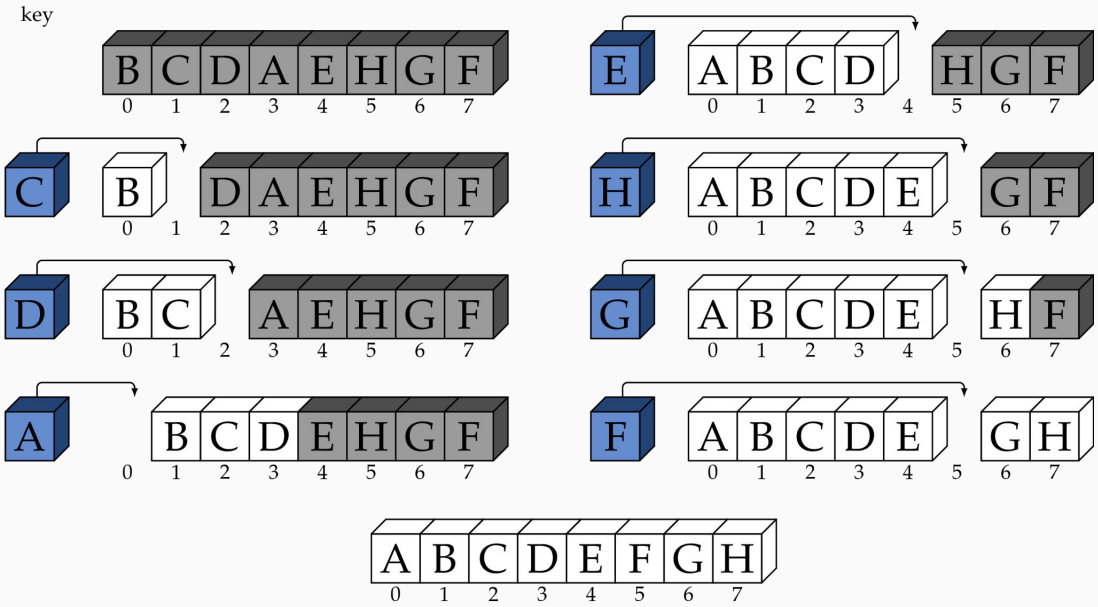


Insertion Sort

"At every step, look at the next object and **insert** it into its correct spot in the sorted sequence you've got so far."

Is this an algorithm?

Maybe a little too vague...



Insertion Sort

Is this an algorithm?

```java
public void sort(int array[]){
    int n = array.length;
    for (int i = 1; i < n; i++){
        int key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
    }
}
```

Insertion Sort

Is this an algorithm?

More accurately a **program**.

```
public void sort(int array[]){
    int n = array.length;
    for (int i = 1; i < n; i++){
        int key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
    }
}
```

Insertion Sort

We can describe algorithms using **pseudocode**.

$$
\begin{array}{l}
\textsc{InsertionSort}(A[1,\ldots,n]): \\
\quad \textbf{for } i = 2,\ldots,n \\
\qquad key := A[i] \\
\qquad j := i - 1 \\
\qquad \textbf{while } j > 0 \text{ and } A[j] > key \\
\qquad\quad A[j+1] := A[j] \\
\qquad\quad j := j - 1 \\
\qquad A[j+1] := key
\end{array}
$$

Insertion Sort

We can describe algorithms using **pseudocode**, i.e. a mix of:

- structure of some programming languages,
- natural languages,
- mathematical notation

$$
\begin{array}{l}
\textsc{InsertionSort}(A[1,\ldots,n]): \\
\quad \textbf{for } i = 2,\ldots,n \\
\qquad key := A[i] \\
\qquad j := i-1 \\
\qquad \textbf{while } j > 0 \text{ and } A[j] > key \\
\qquad\quad A[j+1] := A[j] \\
\qquad\quad j := j-1 \\
\qquad A[j+1] := key
\end{array}
$$

Insertion Sort

```java
public void sort(int array[]){
    int n = array.length;
    for (int i = 1; i < n; i++){
        int key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key){
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
    }
}
```

$\textsc{InsertionSort}(A[1,\ldots,n])$:

$\quad$ **for** $i = 2,\ldots,n$

$\quad\quad$ $key := A[i]$

$\quad\quad$ $j := i - 1$

$\quad\quad$ **while** $j > 0$ and $A[j] > key$

$\quad\quad\quad$ $A[j+1] := A[j]$

$\quad\quad\quad$ $j := j - 1$

$\quad\quad$ $A[j+1] := key$

Design and Analysis

Given a **problem** (e.g. sorting), design an **algorithm** for solving that problem.

Design and Analysis

Given a **problem** (e.g. sorting), design an **algorithm** for solving that problem.

- Prove that it produces the right answer **always**.
- Describe its **worst case** runtime.

Correctness?

Formally showing correctness of recursive algorithms uses **proof by induction**.

$$
\begin{aligned}
&\textsc{MergeSort}(A[1,\ldots,n]):\\
&\qquad \textbf{if } n > 1\\
&\qquad\qquad m := \lfloor n/2 \rfloor\\
&\qquad\qquad \textsc{MergeSort}(A[1,\ldots,m])\\
&\qquad\qquad \textsc{MergeSort}(A[m+1,\ldots,n])\\
&\qquad\qquad \textsc{Merge}(A,m)
\end{aligned}
$$

Runtime?

How long does this algorithm take?

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad\quad key := A[i]\\
&\qquad\quad j := i-1\\
&\qquad\quad \textbf{while } j > 0 \text{ and } A[j] > key\\
&\qquad\qquad\quad A[j+1] := A[j]\\
&\qquad\qquad\quad j := j-1\\
&\qquad\quad A[j+1] := key
\end{aligned}
$$

Runtime?

How long does this algorithm take?

- The **while** loop might need up to ***n*** iterations
- The **for** loop goes for ≈ ***n*** iterations

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad\quad key := A[i]\\
&\qquad\quad j := i-1\\
&\qquad\quad \textbf{while } j > 0 \text{ and } A[j] > key\\
&\qquad\qquad\quad A[j+1] := A[j]\\
&\qquad\qquad\quad j := j-1\\
&\qquad\quad A[j+1] := key
\end{aligned}
$$

Runtime?

How long does this algorithm take?

- The **while** loop might need up to ***n*** iterations
- The **for** loop goes for ≈ ***n*** iterations

Answer: $O(n^2)$

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad\quad key := A[i]\\
&\qquad\quad j := i-1\\
&\qquad\quad \textbf{while } j > 0 \text{ and } A[j] > key\\
&\qquad\qquad\quad A[j+1] := A[j]\\
&\qquad\qquad\quad j := j-1\\
&\qquad\quad A[j+1] := key
\end{aligned}
$$

Runtime?

How long does this algorithm take?

- The **while** loop might need up to ***n*** iterations
- The **for** loop goes for ≈ ***n*** iterations

Answer: $O(n^2)$, but is this the "tightest" analysis?

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad\quad key := A[i]\\
&\qquad\quad j := i-1\\
&\qquad\quad \textbf{while } j > 0 \text{ and } A[j] > key\\
&\qquad\qquad\quad A[j+1] := A[j]\\
&\qquad\qquad\quad j := j-1\\
&\qquad\quad A[j+1] := key
\end{aligned}
$$

Runtime?

$\textsc{MergeSort}(A[1,\ldots,n])$:
    if $n > 1$
        $m := \lfloor n/2 \rfloor$
        $\textsc{MergeSort}(A[1,\ldots,m])$
        $\textsc{MergeSort}(A[m+1,\ldots,n])$
        $\textsc{Merge}(A,m)$

Runtime?

Where does O(***n*** log ***n***) come from for mergesort?

$$
\begin{aligned}
&\textsc{MergeSort}(A[1,\ldots,n]):\\
&\qquad \textbf{if } n > 1\\
&\qquad\qquad m := \lfloor n/2 \rfloor\\
&\qquad\qquad \textsc{MergeSort}(A[1,\ldots,m])\\
&\qquad\qquad \textsc{MergeSort}(A[m+1,\ldots,n])\\
&\qquad\qquad \textsc{Merge}(A,m)
\end{aligned}
$$

Course Overview

Instructor: Prof. Timothy Sun (timothysun@sfsu.edu)

Office Hours: WF 5-6PM on Zoom (starting next week), or by appointment

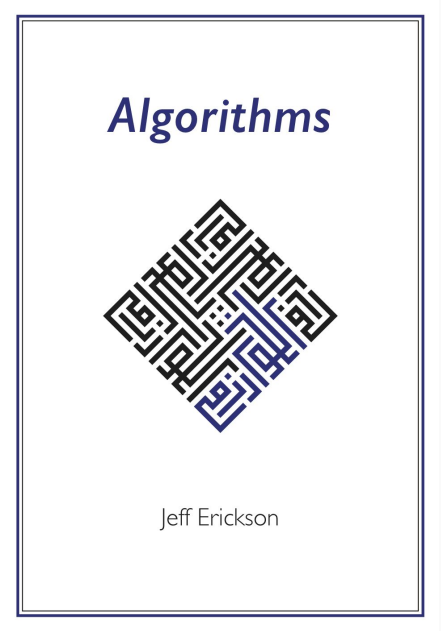
Course Materials: on Canvas

Textbooks

Jeff Erickson, *Algorithms*, 1st Edition.

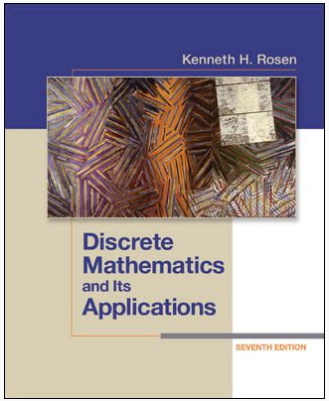
http://jeffe.cs.illinois.edu/teaching/algorithms/

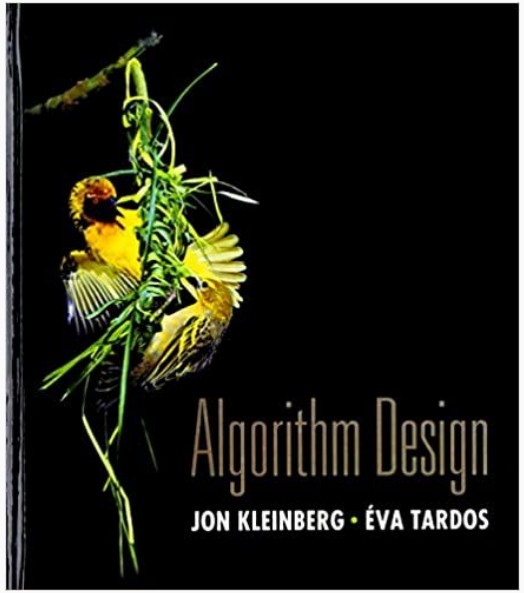
(it's free!)

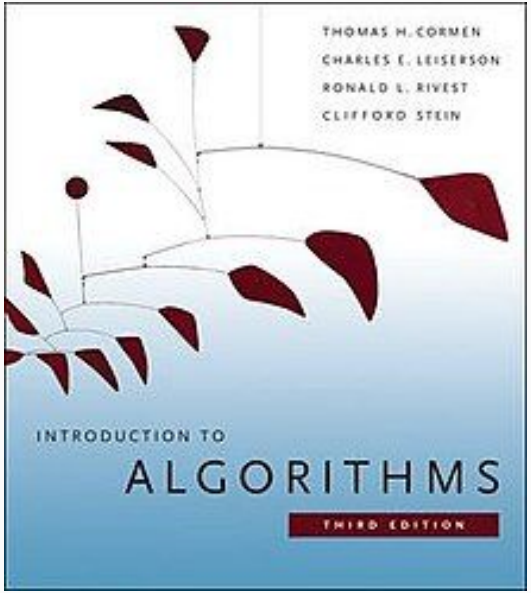


Other Textbook Sources

Handouts to be posted on Canvas (http://canvas.sfsu.edu)







Grading Breakdown

Homework: 35%

Midterm: 25%

Final: 35%

Participation: 5%

Course Overview (cont'd)

Homework (35%):

- Problem sets every two weeks (no programming)
  - Can work with others, but own writing

Course Overview (cont’d)

Midterm (25%):

- In-class

Final (35%):

- During exam period, strong emphasis on later topics

Course Overview (cont’d)

Participation (5%):

- Both in and outside (e.g. Discord) of class

Recursion / Divide and conquer

Many problems have a “recursive structure”: big solution from smaller solutions.

$$
\begin{aligned}
&\textsc{MergeSort}(A[1,\ldots,n]):\\
&\qquad \textbf{if } n > 1\\
&\qquad\qquad m := \lfloor n/2 \rfloor\\
&\qquad\qquad \textsc{MergeSort}(A[1,\ldots,m])\\
&\qquad\qquad \textsc{MergeSort}(A[m+1,\ldots,n])\\
&\qquad\qquad \textsc{Merge}(A,m)
\end{aligned}
$$

Recursion / Divide and conquer

Many problems have a “recursive structure”: big solution from smaller solutions.

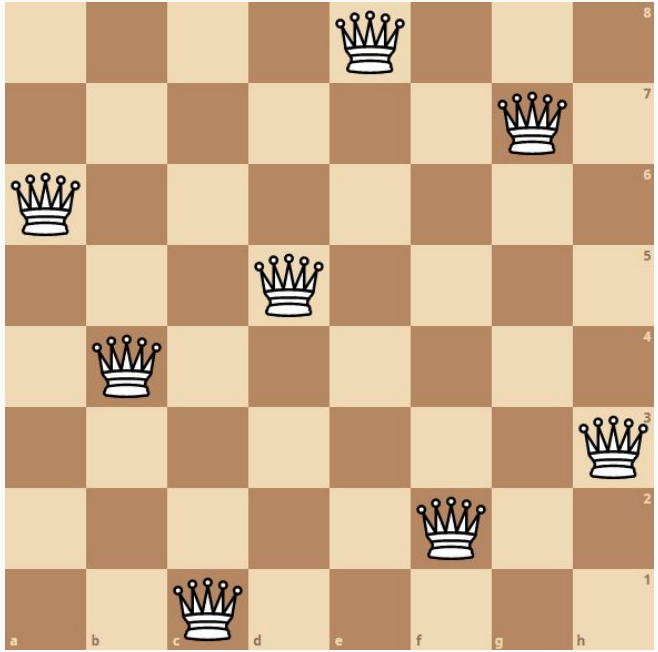
How to design and analyze recursive algorithms?

$$
\begin{aligned}
&\textsc{MergeSort}(A[1,\ldots,n]):\\
&\qquad \textbf{if } n > 1\\
&\qquad\qquad m := \lfloor n/2 \rfloor\\
&\qquad\qquad \textsc{MergeSort}(A[1,\ldots,m])\\
&\qquad\qquad \textsc{MergeSort}(A[m+1,\ldots,n])\\
&\qquad\qquad \textsc{Merge}(A,m)
\end{aligned}
$$

Backtracking

Building up a solution incrementally by guessing and checking.

The **Eight Queens Puzzle** asks whether or not you can place 8 non-attacking queens on a chessboard.



Dynamic Programming

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

What number comes next?

Dynamic Programming

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

What number comes next?

Answer: 144

Dynamic Programming

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

What number comes next?

Answer: 144

This is the **Fibonacci sequence** F_n, defined in the following way:

$F_1 = 1, F_2 = 1$

$F_n = F_{n-1} + F_{n-2}$, for $n > 2$.

Dynamic Programming

$\textsc{Fibonacci}(n)$:

    if $n = 1$ or $n = 2$

        return 1

    else

        return $\textsc{Fibonacci}(n-1) + \textsc{Fibonacci}(n-2)$

Should you use this algorithm?

Dynamic Programming

$$
\begin{aligned}
&\textsc{Fibonacci}(n):\\
&\qquad \textbf{if } n = 1 \text{ or } n = 2\\
&\qquad\qquad \textbf{return } 1\\
&\qquad \textbf{else}\\
&\qquad\qquad \textbf{return } \textsc{Fibonacci}(n-1) + \textsc{Fibonacci}(n-2)
\end{aligned}
$$

Should you use this algorithm? **No, very slow for big *n***

Dynamic Programming

Dynamic programming involves saving intermediate solutions to avoid redundant work.

$\textsc{DPFibonacci}(n)$:

    initialize $F[1, \ldots, \max(2, n)]$
    $F[1] := 1$
    $F[2] := 1$
    for $i = 3, \ldots, n$
        $F[i] := F[i-1] + F[i-2]$
    return $F[n]$

Greedy Algorithms

If you wanted to make change for a certain amount of money, how would you do it to produce the fewest number of coins?

Greedy Algorithms

If you wanted to make change for a certain amount of money, how would you do it to produce the fewest number of coins?

Take the biggest coin possible.

Greedy Algorithms

“Cashier’s algorithm” for making change:

- Take quarters until change is < 25¢
- Take dimes until change is < 10¢
- Take nickels until change is < 5¢
- Take pennies for the rest.

Greedy Algorithms

“Cashier’s algorithm” for making change:

- Take quarters until change is < 25¢
- Take dimes until change is < 10¢
- Take nickels until change is < 5¢
- Take pennies for the rest.

Always the least number of coins!

Cashier’s algorithm

Why is the cashier’s algorithm not optimal for 1¢, 5¢, 7¢?

Cashier’s algorithm

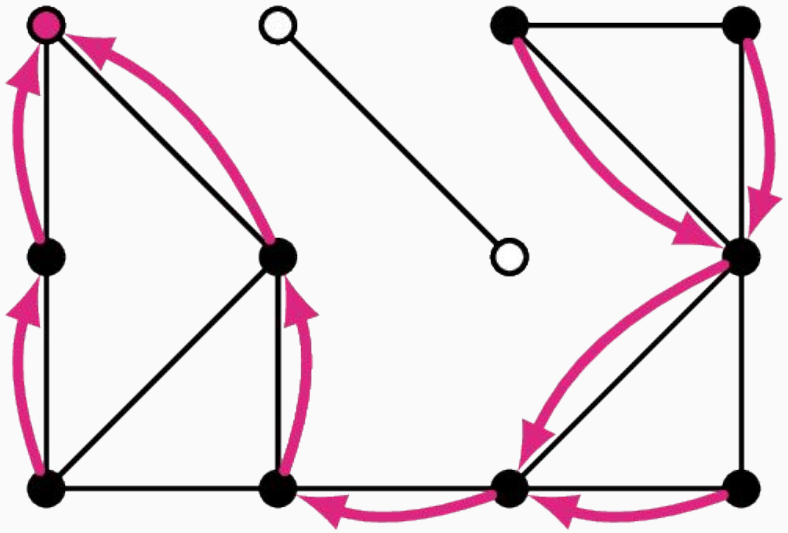
Why is the cashier’s algorithm not optimal for 1¢, 5¢, 7¢?

10 = 5 + 5 (optimal) = 7 + 1 + 1 + 1 (greedy)

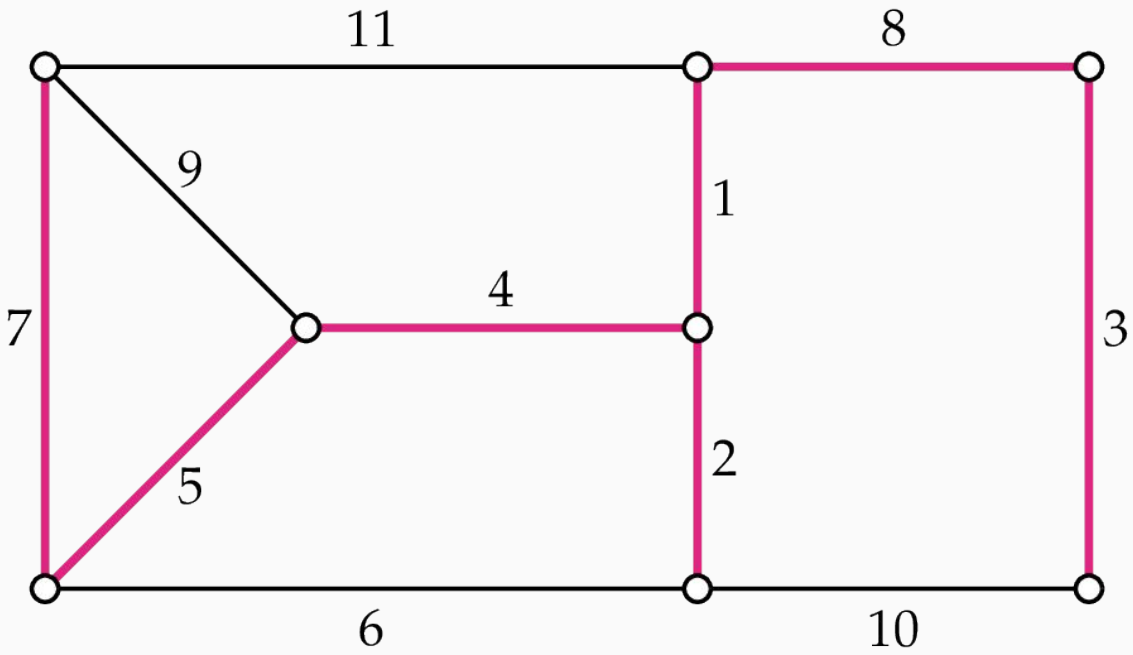
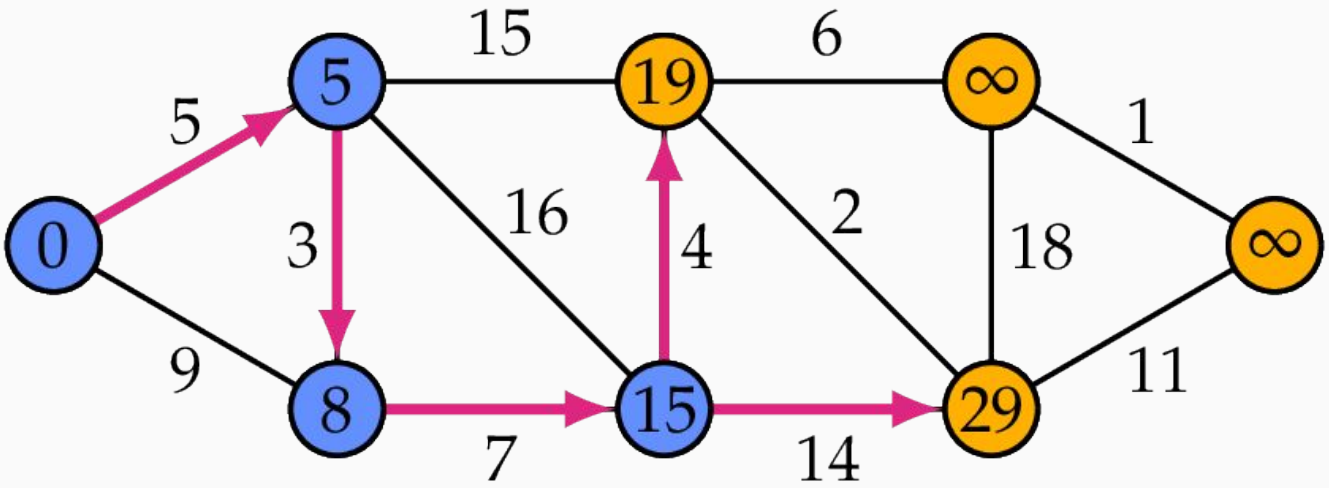
Graph Algorithms

Graphs are a fundamental data structure in computer science.

Using aforementioned methods for problems involving graphs.



Minimum Spanning Trees



Shortest Paths